

Proyecto modulo 2 IoT

2024-1

ASEGURAMIENTO DE UN SISTEMA IOT

Kevin Julián Chavarria Olarte

Kevinchavarria257628@correo.itm.edu.co

Instituto Tecnológico Metropolitano Medellín, Colombia

Abstract: Una pequeña empresa tiene implementado un sistema IoT, sin embargo, este se encuentra vulnerable y sin protección a posibles ataques. El propietario de la empresa quiere instalar un esquema de seguridad, pero no sabe cómo hacerlo. Se ha puesto en contacto con usted quien tiene una empresa que puede diseñar e instalar el sistema. Su especialidad se centra en la ciberseguridad de todo el proceso de diseño, aprovisionamiento y desarrollo del sistema. Por tal motivo usted debe realizar este proceso desde la implementación de la red y dispositivos, análisis de vulnerabilidades, simulación de ataques e implementación de la seguridad.

I. Introducción

para este proyecto se hará una simulación donde se mostrarán los pasos a realizar para la implementación de una red y dispositivos, análisis de vulnerabilidades, simulación de ataques e implementación de la seguridad. Para esto se hará uso de una placa física ESP8266 ya configurada con un código de conexión mqtt a una plataforma en la nube (ubidots) y meta explotada (ya que estas placas tienen una buena protección y seguridad); y de una maquina virtual (IoTsec-kali) para así poder realizar los análisis de vulnerabilidades, simulación de ataques e implementación de la seguridad de esta.

II. Objetivos

- 1) Implementar la red mediante máquinas virtuales y la placa física (ESP-8266) integrando el proyecto Nro 1 (sistema de control con ESP-8266 con sistema de monitoreo externo a través de una plataforma en la nube).

- 2) Realizar el proceso de análisis de vulnerabilidades para determinar los bugs, puertas traseras de la placa.
- 3) Explicar al menos 4 vulnerabilidades halladas, en que consisten y como se podría proteger para evitar ser atacado.
- 4) Proponer e implementar una forma de acceder (ataque) a la placa mediante la explotación de alguna de las vulnerabilidades encontradas en el paso anterior y explicar el paso a paso.
- 5) Proponer e implementar formas de protección (hardening) para la red de dispositivos IoT.

III. Procedimiento

- 1) Se implemento la red de la máquina virtual Kali-Linux(IoTsec-kali) configurando la red como adaptador de puente y luego asignarle una dirección ip de la red usando el código `sudo dhclient -v`. [1]

```
root@kali:~# sudo dhclient -v
Internet Systems Consortium DHCP Client 4.3.5
Copyright 2004-2016 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/eth0/08:00:27:a7:a2:2a
Sending on LPF/eth0/08:00:27:a7:a2:2a
Sending on Socket/fallback
DHCPREQUEST of 192.168.1.8 on eth0 to 255.255.255.255 port 67
DHCPCACK of 192.168.1.8 from 192.168.1.254
smbd.service is not active, cannot reload.
invoke-rc.d: initscript smbd, action "reload" failed.
bound to 192.168.1.8 -- renewal in 239402 seconds.
```

Figura 1.

En la figura uno se puede observar que la Kali tomo una dirección ip para su red la cual es 192.168.1.8 en la eth0.

A continuación, se mostrará el código usado para cargarle a la placa física.

```
#include <ESP8266WiFi.h>

#include <ESP8266WebServer.h>

#include <PubSubClient.h>

#include <DHT.h>

#include <Ubidots.h>

#include <Servo.h>

#include <FS.h>

#define servo_topic "servo"

#define servo_pin D0

#define DHTPIN D2

#define DHTTYPE DHT11

#define ldrPin A0 // Pin del LDR

#define led D1

// Configuración de la red WiFi

const char* ssid = "FLA OLARTE";
```

```
const char* password =
"minutointerneta1000";
```

// Credenciales de Ubidots (MQTT)

```
const char* mqttServer =
"industrial.api.ubidots.com";

const int mqttPort = 1883;

const char* ubidotsToken = "BBUS-
b8B11TyLThh470jnBg0bGpCOcfEElp";

const char* mqttPassword = "BBUS-
8dce9b11a78b8860aa11355d3ff6134356f
";
```

```
unsigned long previousDHTMillis = 0;

unsigned long previousServoMillis = 0;

unsigned long previousLDRMillis = 0;

const long DHTInterval = 60000; //
Intervalo para enviar datos del DHT11 (1
minuto)

const long servoInterval = 1000; //
Intervalo para enviar datos del servo
(1000 ms)

const long ldrInterval = 1000; // Intervalo
para enviar datos del LDR (1000 ms)
```

```
WiFiClient espClient; // Usar
WiFiClientSecure para TLS

PubSubClient client(espClient);

Ubidots ubidots("BBUS-
b8B11TyLThh470jnBg0bGpCOcfEElp");

DHT dht(DHTPIN, DHTTYPE);
```

```

Servo servo1;

ESP8266WebServer server(80); // Usar el
puerto 443 para HTTPS

int pulmin = 700;

int pulmax = 2500;

int servo;

float lastDistance = 0.0;

bool doorOpen = false;

const int thresholdHigh = 80;

int lastLDRValue = 0;

int lastSentLDRValue = 0;

bool manualMode = false; // Variable para
controlar el modo manual o automático
del servo

float currentTemperature = NAN;

float currentHumidity = NAN;

int currentLDRValue = 0;

int currentServoValue = 0;

bool currentDoorState = false;

String message = "";

// Variables para autenticación básica
const char* www_username = "admin";
const char* www_password = "1234";

// Función para validar las credenciales de
la web
bool is_authenticated() {

```

```

    if (!server.authenticate(www_username,
www_password)) {
        server.requestAuthentication();

        return false;
    }

    return true;
}

void callback(char* topic, byte* payload,
unsigned int length) {
    Serial.print("Mensaje recibido [");
    Serial.print(topic);
    Serial.print("] ");
    message = "";
    for (int i = 0; i < length; i++) {
        message += (char)payload[i];
    }
    Serial.print("Contenido: ");
    Serial.println(message);

    if (String(topic) ==
"v1.6/devices/ESP8266/servo") {
        int servoValue = message.toInt();

        Serial.print("Orden de posicionamiento
del servo recibida: ");

        Serial.println(servoValue);
        servo1.write(servoValue);
        currentServoValue = servoValue;
    }
}

```

```

void reconnect() {
    while (!client.connected()) {
        Serial.print("Intentando conexión
MQTT...");

        if (client.connect("ESP8266Client",
ubidotsToken, "")) { // Usar token como
usuario, sin contraseña

            Serial.println("Conectado");

client.subscribe("/v1.6/devices/ESP8266/
servo"); // Suscribirse al tema correcto

        } else {
            Serial.print("falló, rc=");

            Serial.print(client.state());

            Serial.println(" Intenta de nuevo en 5
segundos");

            delay(5000);
        }
    }
}

void handleRoot() {
    if (!is_authenticated()) {
        return;
    }

    String html = "<html><head><meta
charset=\"UTF-8\"><title>ESP8266 Web
Server</title></head><body>";

```

```

        html += "<h1>MQTT Monitor</h1>";

        html += "<p>Último mensaje MQTT
recibido: ";

        html += message;

        html += "</p>";

        html += "<h2>Datos actuales</h2>";

        html += "<p>Temperatura: " +
String(currentTemperature) + " °C</p>";

        html += "<p>Humedad: " +
String(currentHumidity) + " %</p>";

        html += "<p>LDR: " +
String(currentLDRValue) + "</p>";

        html += "<p>Servo: " +
String(currentServoValue) + "</p>";

        html += "<h2>Control del Servo</h2>";

        html += "<form action=\"/send\"
method=\"POST\">";

        html += "Posición del Servo (0-180):
<input type=\"number\"
name=\"servoPos\" min=\"0\"
max=\"180\">";

        html += "<input type=\"submit\"
value=\"Enviar\">";

        html += "</form>";

        html += "<h2>Modo de
operación</h2>";

        html += "<form action=\"/mode\"
method=\"POST\">";

        html += "<input type=\"radio\"
name=\"mode\" value=\"manual\" ";

        html += manualMode ? "checked" : "";

        html += "> Manual<br>";

```

```

html += "<input type=\"radio\"
name=\"mode\" value=\"automatic\" ";
html += !manualMode ? "checked" : "";
html += "> Automático<br>";

```

```

html += "<input type=\"submit\"
value=\"Cambiar modo\">";
html += "</form>";
html += "</body></html>";

```

```

server.send(200, "text/html", html);
}

```

```

void handleSend() {
    String servoPos =
server.arg("servoPos");
    if (servoPos.length() > 0) {
        int newServoValue = servoPos.toInt();
        if (newServoValue >= 0 &&
newServoValue <= 180) {
            if (newServoValue !=
currentServoValue) { // Verifica si hay un
cambio en la posición del servo
                servo1.write(newServoValue);
                currentServoValue =
newServoValue; // Actualiza la posición
actual del servo
            }
        }
    }
}

```

```

client.publish("v1.6/devices/ESP8266/ser
vo", String(newServoValue).c_str());

```

```

        message = "Servo position set to: " +
String(newServoValue); // Actualiza el
mensaje MQTT recibido
    }
}
}

```

```

server.sendHeader("Location", "/");
server.send(303);
}

```

```

void handleMode() {
    if (!is_authenticated()) {
        return;
    }

    String mode = server.arg("mode");
    if (mode == "manual") {
        manualMode = true;
    } else if (mode == "automatic") {
        manualMode = false;
    }
}

```

```

server.sendHeader("Location", "/");
server.send(303);
}

```

```

void setup() {
    Serial.begin(115200);
}

```

```

servo1.attach(servo_pin, pulmin,
pulmax);

Serial.println();

Serial.print("Conectando a ");

Serial.println(ssid);

WiFi.begin(ssid, password);

while (WiFi.status() !=
WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("");
Serial.println("WiFi conectado");
Serial.println("Dirección IP: ");
Serial.println(WiFi.localIP());

server.on("/", handleRoot);

server.on("/send", HTTP_POST,
handleSend);

server.on("/mode", HTTP_POST,
handleMode);

server.begin();

Serial.println("Servidor web iniciado");

client.setServer(mqttServer, mqttPort);

client.setCallback(callback);

```

```

dht.begin();

pinMode(ldrPin, INPUT);
}

void loop() {
    if (!client.connected()) {
        reconnect();
    }

    client.loop();

    server.handleClient(); // Manejar las
solicitudes del servidor web

    unsigned long currentMillis = millis();

    // Enviar datos del DHT11 cada 1
minuto

    if (currentMillis - previousDHTMillis
>= DHTInterval) {
        previousDHTMillis = currentMillis;

        currentTemperature =
dht.readTemperature();

        if (!isnan(currentTemperature)) {
            Serial.print("Temperatura: ");
            Serial.println(currentTemperature);

            ubidots.add("temperature",
currentTemperature);

            ubidots.send("temperature"); // Envía
los datos de la variable "temperature" a
Ubidots

        }

        currentHumidity = dht.readHumidity();

```

```

if (!isnan(currentHumidity)) {
    Serial.print("Humedad: ");
    Serial.println(currentHumidity);
    ubidots.add("humedad",
currentHumidity);

    ubidots.send("temperature"); // Envía
los datos de la variable "humedad" a
Ubidots

}
}

```

```

// Controlar el servo en modo
automático

if (!manualMode) {
    if (currentMillis - previousLDRMillis
>= ldrInterval) {

        previousLDRMillis = currentMillis;

        currentLDRValue =
analogRead(ldrPin);

        if (abs(currentLDRValue -
lastLDRValue) > thresholdHigh ||
abs(currentLDRValue -
lastSentLDRValue) > thresholdHigh/2) {

            int lumen = map(currentLDRValue,
0, 1023, 0, 1000); // Convertir valor del
LDR a lúmenes

            ubidots.add("lumen", lumen);

            ubidots.send("temperature"); //
Envía los datos de luz a Ubidots

            lastSentLDRValue =
currentLDRValue;

        }
    }
}

```

```

        if (currentLDRValue > thresholdHigh)
{ // Si la luz es alta

            servo1.write(0); // Mover servo a 0
grados

        } else { // Si la luz es baja

            servo1.write(90); // Mover servo a
90 grados

        }

    }

}

```

```

// Enviar datos del servo cuando sea
necesario

if (currentMillis - previousServoMillis
>= servoInterval) {

    previousServoMillis = currentMillis;

    currentServoValue = servo1.read();

    static int lastServoValue =
currentServoValue; // Declaración de la
variable lastServoValue

    if (currentServoValue !=
lastServoValue) {

        ubidots.add("servo_value",
currentServoValue); //

        ubidots.send("temperature"); // Envía
los datos del servo a Ubidots

        lastServoValue = currentServoValue;

    }

    Serial.print("Posicion actual del servo:
");

    Serial.println(currentServoValue);

    Serial.print("Fuente de la orden: ");

```

```

Serial.println(manualMode ? "Manual"
: "Automático");

Serial.print("Valor del LDR: ");

Serial.println(currentLDRValue);

}

} [1]

```

- 2) Se realizo un escaneo de la red especificando la dirección de red y la máscara de bits haciendo uso del código nmap 192.168.1.0/24 en la Kali-Linux. [2]

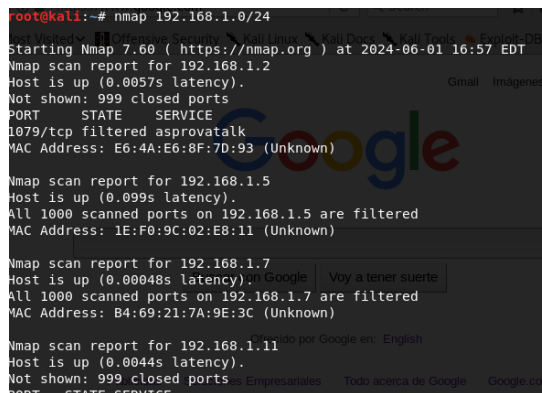


Figura 2.

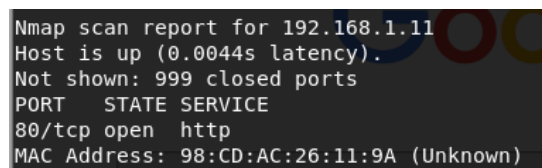


Figura 3.

En la figura 2 se pueden observar el escaneo realizado con la herramienta nmap en la red donde se muestra dispositivos especificando la dirección de red y la máscara de bits, y en la figura 3 se puede observar que uno de los dispositivos tiene un puerto abierto que es el 80/tcp el cual es el servicio de http y dirección MAC: 98:CD:AC:26:11:9A (desconocido).

Con el escaneo anterior ya se pudo encontrar una vulnerabilidad en la red en uno de los dispositivos, con la cual al mismo tiempo identificamos cual era nuestra placa.

Se realizo otro escaneo de la red usando *nmap -sV 192.168.1.0/24* la cual se para detectar versiones específicas de los servicios que se ejecutan en los puertos abiertos. Esto ayuda a identificar vulnerabilidades específicas asociadas con esas versiones. [2]

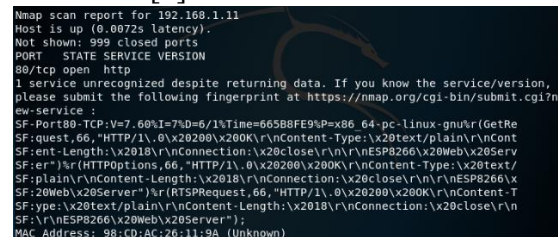


Figura 4.

En la figura 4 podemos observar que:

El host está activo (latencia de 0.0072s).

Puerto 80/tcp está abierto, corriendo un servicio HTTP.

El servicio se identifica como "ESP8266 Web Server".

Dirección MAC: 98:CD:AC:26:11:9A (desconocido).

Luego se utilizo el siguiente código *sudo nmap --script ftp-proftpd-backdoor.nse,ftp-vsftpd-backdoor.nse,http-dlink-backdoor.nse,irc-unrealircd-backdoor.nse,smb-double-pulsar-backdoor.nse 192.168.1.11* para buscar puertas traseras conocidas en ese dispositivo. [2]


```

root@kali:~# sudo nmap --script ftp-proftpd-backdoor.nse,ftp-vsftpd
,http-dlink-backdoor.nse,irc-unrealircd-backdoor.nse,smb-double-pul
nse 192.168.1.11

Starting Nmap 7.60 ( https://nmap.org ) at 2024-06-01 17:37 EDT
Nmap scan report for 192.168.1.11
Host is up (0.0039s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 98:CD:AC:26:11:9A (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 10.59 seconds

```

Figura 5.

En la figura 5 podemos observar el resultado del escaneo y nos muestra que no se detectaron puertas traseras conocidas para los servicios FTP ProFTPD, FTP VSFTPD, HTTP D-Link, UnrealIRCd IRC, o SMB DoublePulsar en esta dirección IP. Esto sugiere que el dispositivo podría no estar comprometido con ninguna de estas puertas traseras específicas.

- 3) Se uso la herramienta nikto para visualizar las vulnerabilidades halladas usando el código *nikto -h <http://192.168.1.11>*. [1]

```

root@kali:~# nikto -h http://192.168.1.10
+ Nikto v2.1.6
+-----+
+ Target IP:      192.168.1.10
+ Target Hostname: 192.168.1.10
+ Target Port:    80
+ Start Time:     2024-06-05 16:23:07 (GMT-4)
+-----+
+ Server: No banner retrieved
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the
gent to protect against some forms of XSS
+ The X-Content-Type-Options header is not set. This could allow the user
to render the content of the site in a different fashion to the MIME type
+ / - Requires Authentication for realm 'Login Required'
+ Default account found for 'Login Required' at / (ID 'admin', PW '1234').
ic account discovered.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Web Server returns a valid response with junk HTTP methods, this may cau
se positives.
+ /666%0a%0a<script>alert('Vulnerable');</script>666.jsp: Apache Tomcat 4.
nux is vulnerable to Cross Site Scripting (XSS). http://www.cert.org/advis

```

Figura 6.

En la figura 6 se muestran las vulnerabilidades encontradas con la herramienta de nikto las cuales fueron:

- Falta de la cabecera X-Frame-Options

La cabecera X-Frame-Options evita que su contenido sea embebido en un iframe de otro sitio web, lo que protege contra ataques de clickjacking.

Para proteger este y evitar se atacado Se debe configurar el servidor web para incluir el encabezado X-Frame-Options con el valor "DENY" o "SAMEORIGIN" para evitar que se cargue la página en un marco externo.

- Falta de la cabecera X-XSS-Protection

La cabecera X-XSS-Protection activa el filtro de protección contra ataques XSS (Cross-Site Scripting) en navegadores compatibles.

Para proteger este se debe configurar el servidor web para incluir el encabezado X-XSS-Protection con el valor "1; mode=block" para habilitar la protección del navegador contra ataques XSS.

- Falta de la cabecera X-Content-Type-Options

La cabecera X-Content-Type-Options evita que el navegador interprete el contenido en un tipo diferente al especificado en el Content-Type.

Para proteger esta se debe añadir la cabecera X-Content-Type-Options con el valor nosniff para evitar la interpretación incorrecta del contenido. [1]

- Al ingresar el usuario, este no posee una forma de cerrar la sesión.
- También hay una autenticación con una contraseña débil.
- No posee un numero limitado de intentos en la autenticación.
- No usa una http segura donde se ve relacionada información importante y manipulación de de un servo motor.

Las medidas de protección sugeridas para estas es implementar un usuario y contraseña mas segura, implementar un numero limitados de intentos de acceso y una forma de cerrar la sesión del usuario; también se sugiere implementar una http segura la cual requiere de un certificado e implementar este al código de la placa.

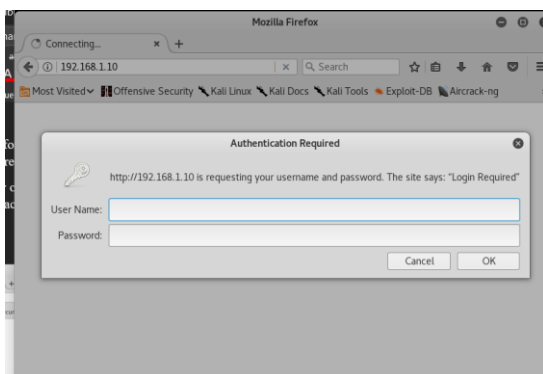


Figura 7.

En la figura 7 se puede observar que al intentar entrar al servidor web de la placa aparece la autenticación requerida.

- 4) Ahora se realiza un ataque, en este caso se realizara un ataque de fuerza bruta para conseguir dicha autenticación y acceder al servidor web usando el código *de hydra*.

Si inicialmente ya el atacante conoce el usuario de alguna manera podria usar el código de esta manera: *hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.1.10 http-get /* [3]

Si no conoces el nombre de usuario, necesitarás usar una lista de posibles nombres de usuario junto con una lista de contraseñas para realizar un ataque de fuerza bruta. Hydra soporta

este tipo de ataques utilizando la opción -L para especificar una lista de nombres de usuario. [1]

Crear una Lista de Nombres de Usuario:
echo -e "admin\nuser\nnguest\nroot\ntest"
> /usr/share/wordlists/usernames.txt. [1]

Luego ejecuta hydra con la lista de nombres de usuarios: *hydra -L /usr/share/wordlists/usernames.txt -P /usr/share/wordlists/rockyou.txt 192.168.1.10 http-get /* o usar una lista que ya posee la Kali-linux: *hydra -L /usr/share/wordlists/usernames.txt -P /usr/share/wordlists/rockyou.txt 192.168.1.10 http-get /*

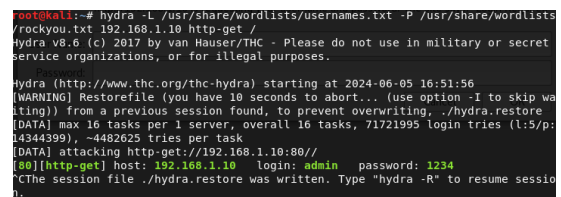


Figura 8.



Figura 9.

En la figura 8 se puede observar que el ataque descubrió un usuario y contraseña; en la figura 9 se observa que se logro entrar usando dichas credenciales donde

existen unos datos y manipulación del motor

Figura 10.

- 5) Para mejorar la seguridad (hardening) de la red de dispositivos IoT y abordar las vulnerabilidades encontradas por Nikto en tu ESP8266, se puede seguir los siguientes pasos:

Modificar el código del ESP8266 para incluir las cabeceras de seguridad necesarias:

Evita ataques de clickjacking añadiendo la cabecera X-Frame-Options:

```
void handleRoot() {  
    server.sendHeader("X-Frame-Options",  
        "DENY"); [4]  
    server.send(200, "text/plain", "ESP8266  
Web Server");  
}
```

X-XSS-Protection

Protege contra algunos tipos de ataques XSS añadiendo la cabecera X-XSS-Protection:

```
void handleRoot() {  
    server.sendHeader("X-XSS-Protection",  
        "1; mode=block"); [5]  
    server.send(200, "text/plain", "ESP8266  
Web Server");  
}
```

X-Content-Type-Options

Evita que el navegador interprete los archivos de forma incorrecta añadiendo la cabecera X-Content-Type-Options:

```
void handleRoot() {  
    server.sendHeader("X-Content-Type-  
Options", "nosniff") [6];  
    server.send(200, "text/plain", "ESP8266  
Web Server");  
}
```

La implementación completa de esto sería algo así:

```
void handleRoot() {  
    server.sendHeader("X-Frame-Options",  
        "DENY");  
    server.sendHeader("X-XSS-Protection",  
        "1; mode=block");  
    server.sendHeader("X-Content-Type-  
Options", "nosniff");  
    server.send(200, "text/plain", "ESP8266  
Web Server");  
}
```

Para mejorar la seguridad de las comunicaciones, considera usar HTTPS. Para implementar HTTPS en el ESP8266, Para implementar la http segura se necesita un certificado SSL/TLS para tu dominio. Puedes obtener uno a través de una autoridad de certificación (CA) confiable, o puedes generar uno tú mismo con herramientas como OpenSSL para uso local o de desarrollo. [1]

En el siguiente código se muestra como se implementaría el hardening a la placa en el código incluyendo la https. Sin embargo, debido a las restricciones necesarias para la obtención de una https,

se seguirá utilizando el puerto 80 para abordar los siguientes puntos de este proyecto.

```
#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>
#include <ESP8266WebServerSecure.h>
```

// Configurar tu servidor HTTPS en lugar de HTTP

ESP8266WebServerSecure server(443); // Cambia al puerto 443 para HTTPS

```
void setup() {
    // Código de configuración
    server.begin();
}
```

Autenticación y Autorización

Implementa un sistema de autenticación con contraseñas seguras para limitar el acceso al servidor web, así como un método de cierre de sesión:

```
const char* www_username =
"espconhardening";
```

```
const char* www_password =
"c0ntr@sena";
```

```
bool isAuthenticated = false;
```

```
int failedAttempts = 0;
```

```
const int maxAttempts = 3; // Número
máximo de intentos de autenticación
```

// Función para validar las credenciales de la web

```
bool is_authenticated() {
    if (!isAuthenticated) {
        if
(!server.authenticate(www_username,
www_password)) {
            failedAttempts++;
            if (failedAttempts >= maxAttempts) {
                server.send(403, "text/plain",
                "Acceso bloqueado por múltiples intentos
fallidos.");
                return false;
            }
            server.requestAuthentication();
            return false;
        } else {
            isAuthenticated = true;
            failedAttempts = 0; // Reiniciar
            contador de intentos fallidos después de
            una autenticación exitosa
        }
    }
    return true;
}
```

```
void handleLogout() {
    isAuthenticated = false;
    server.sendHeader("Location", "/");
    server.send(303);
}
```

Deshabilitar Servicios Innecesarios

Desactiva servicios y puertos no necesarios para reducir la superficie de ataque.

El código siguiente sería el código implementado para realizar el hardening mencionado anteriormente:

```
#include <ESP8266WiFi.h>

#include <ESP8266WebServer.h>

#include <PubSubClient.h>

#include <DHT.h>

#include <Ubidots.h>

#include <Servo.h>

#include <FS.h>


#define servo_topic "servo"

#define servo_pin D0

#define DHTPIN D2

#define DHTTYPE DHT11


#define ldrPin A0 // Pin del LDR

#define led D1


// Configuración de la red WiFi

const char* ssid = "FLA OLARTE";

const char* password =
"minutointerneta1000";


// Credenciales de Ubidots (MQTT)
```

```
const char* mqttServer =
"industrial.api.ubidots.com";

const int mqttPort = 1883;

const char* ubidotsToken = "BBUS-
b8B11TyLThh470jnBg0bGpCOcfEElp";

const char* mqttPassword = "BBUS-
8dce9b11a78b8860aa11355d3ff6134356f
";


unsigned long previousDHTMillis = 0;

unsigned long previousServoMillis = 0;

unsigned long previousLDRMillis = 0;

const long DHTInterval = 60000; //
Intervalo para enviar datos del DHT11 (1
minuto)

const long servoInterval = 1000; //
Intervalo para enviar datos del servo
(1000 ms)

const long ldrInterval = 1000; //
Intervalo para enviar datos del LDR
(1000 ms)


WiFiClient espClient; // Usar
WiFiClientSecure para TLS

PubSubClient client(espClient);

Ubidots ubidots("BBUS-
b8B11TyLThh470jnBg0bGpCOcfEElp");


DHT dht(DHTPIN, DHTTYPE);

Servo servo1;

ESP8266WebServer server(80); // Usar el
puerto 443 para HTTPS
```

```

int pulmin = 700;
int pulmax = 2500;
int servo;
float lastDistance = 0.0;
bool doorOpen = false;
const int thresholdHigh = 80;
int lastLDRValue = 0;
int lastSentLDRValue = 0;
bool manualMode = false; // Variable
para controlar el modo manual o
automático del servo

float currentTemperature = NAN;
float currentHumidity = NAN;
int currentLDRValue = 0;
int currentServoValue = 0;
bool currentDoorState = false;

String message = "";
// Variables para autenticación básica
const char* www_username =
"espconhardening";
const char* www_password =
"c0ntr@sena";
bool isAuthenticated = false;
int failedAttempts = 0;
const int maxAttempts = 3; // Número
máximo de intentos de autenticación

```

```

// Función para validar las credenciales
de la web
bool is_authenticated() {
    if (!isAuthenticated) {
        if
(!server.authenticate(www_username,
www_password)) {
            failedAttempts++;
            if (failedAttempts >= maxAttempts) {
                server.send(403, "text/plain",
"Acceso bloqueado por múltiples intentos
fallidos.");
                return false;
            }
            server.requestAuthentication();
            return false;
        } else {
            isAuthenticated = true;
            failedAttempts = 0; // Reiniciar
contador de intentos fallidos después de
una autenticación exitosa
        }
    }
    return true;
}

void handleLogout() {
    isAuthenticated = false;
    server.sendHeader("Location", "/");
    server.send(303);
}

```

```
void callback(char* topic, byte* payload,  
unsigned int length) {
```

```
    Serial.print("Mensaje recibido [");
```

```
    Serial.print(topic);
```

```
    Serial.print("] ");
```

```
    String message = "";
```

```
    for (int i = 0; i < length; i++) {
```

```
        message += (char)payload[i];
```

```
    }
```

```
    Serial.print("Contenido: ");
```

```
    Serial.println(message);
```

```
    if (String(topic) ==  
    "v1.6/devices/ESP8266/servo") {
```

```
        int servoValue = message.toInt();
```

```
        Serial.print("Orden de posicionamiento  
del servo recibida: ");
```

```
        Serial.println(servoValue);
```

```
        servo1.write(servoValue);
```

```
        currentServoValue = servoValue;
```

```
    }
```

```
}
```

```
void reconnect() {
```

```
    while (!client.connected()) {
```

```
        Serial.print("Intentando conexión  
MQTT...");
```

```
        if (client.connect("ESP8266Client",  
ubidotsToken, "")) { // Usar token como  
usuario, sin contraseña
```

```
            Serial.println("Conectado");
```

```
            client.subscribe("/v1.6/devices/ESP8266/s  
ervo"); // Suscribirse al tema correcto
```

```
        } else {
```

```
            Serial.print("falló, rc=");
```

```
            Serial.print(client.state());
```

```
            Serial.println(" Intenta de nuevo en 5  
segundos");
```

```
            delay(5000);
```

```
        }
```

```
    }
```

```
}
```

```
void handleRoot() {
```

```
    if (!is_authenticated()) {
```

```
        return;
```

```
    }
```

```
    String html = "<html><head><meta  
charset=\"UTF-8\"><title>ESP8266  
Web Server</title></head><body>";
```

```
    html += "<h1>MQTT Monitor</h1>";
```

```
    html += "<p>Último mensaje MQTT  
recibido: ";
```

```
    html += message;
```

```
    html += "</p>";
```

```
    html += "<h2>Datos actuales</h2>";
```

```

    html += "<p>Temperatura: " +
String(currentTemperature) + " °C</p>";

    html += "<p>Humedad: " +
String(currentHumidity) + " %</p>";

    html += "<p>LDR: " +
String(currentLDRValue) + "</p>";

    html += "<p>Servo: " +
String(currentServoValue) + "</p>";

    html += "<h2>Control del
Servo</h2>";

    html += "<form action=\\\"/send\\\"
method=\\\"POST\\\">";

    html += "Posición del Servo (0-180):
<input type=\\\"number\\\"
name=\\\"servoPos\\\" min=\\\"0\\\"
max=\\\"180\\\">";

    html += "<input type=\\\"submit\\\"
value=\\\"Enviar\\\">";

    html += "</form>";

    html += "<h2>Modo de
operación</h2>";

    html += "<form action=\\\"/mode\\\"
method=\\\"POST\\\">";

    html += "<input type=\\\"radio\\\"
name=\\\"mode\\\" value=\\\"manual\\\" ";

    html += manualMode ? "checked" : "";

    html += "> Manual<br>";

    html += "<input type=\\\"radio\\\"
name=\\\"mode\\\" value=\\\"automatic\\\" ";

    html += !manualMode ? "checked" : "";

    html += "> Automático<br>";

    html += "<input type=\\\"submit\\\"
value=\\\"Cambiar modo\\\">";

    html += "</form>";

```

```

    html += "<h2>Sesión</h2>";

    html += "<form action=\\\"/logout\\\"
method=\\\"POST\\\">";

    html += "<input type=\\\"submit\\\"
value=\\\"Cerrar Sesión\\\">";

    html += "</form>";

    html += "</body></html>";

    // Añadir cabeceras de seguridad

    server.setHeader("X-Frame-Options",
"DENY");

    server.setHeader("X-XSS-Protection",
"1; mode=block");

    server.setHeader("X-Content-Type-
Options", "nosniff");

    server.setHeader("Server",
"SecureServer"); // Cambiar el nombre
del servidor

    server.send(200, "text/html", html);
}

void handleSend() {
    if (!is_authenticated()) {
        return;
    }

    String servoPos =
server.arg("servoPos");

    if (servoPos.length() > 0) {
        int newServoValue = servoPos.toInt();

        if (newServoValue >= 0 &&
newServoValue <= 180) {

```



```
    if (newServoValue !=  
currentServoValue) { // Verifica si hay un  
cambio en la posición del servo
```

```
servo1.write(newServoValue);
```

```
currentServoValue =  
newServoValue; // Actualiza la posición  
actual del servo
```

```
client.publish("v1.6/devices/ESP8266/ser  
vo", String(newServoValue).c_str());
```

```
message = "Servo position set to: "  
+ String(newServoValue); // Actualiza el  
mensaje MQTT recibido
```

```
}
```

```
}
```

```
}
```

```
server.sendHeader("Location", "/");
```

```
server.send(303);
```

```
}
```

```
void handleMode() {
```

```
if (!is_authenticated()) {
```

```
return;
```

```
}
```

```
String mode = server.arg("mode");
```

```
if (mode == "manual") {
```

```
manualMode = true;
```

```
} else if (mode == "automatic") {
```

```
manualMode = false;
```

```
}
```

```
server.sendHeader("Location", "/");
```

```
server.send(303);
```

```
}
```

```
void setup() {
```

```
Serial.begin(115200);
```

```
servo1.attach(servo_pin, pulmin,  
pulmax);
```

```
Serial.println();
```

```
Serial.print("Conectando a ");
```

```
Serial.println(ssid);
```

```
WiFi.begin(ssid, password);
```

```
while (WiFi.status() !=  
WL_CONNECTED) {
```

```
delay(500);
```

```
Serial.print(".");
```

```
}
```

```
Serial.println("");
```

```
Serial.println("WiFi conectado");
```

```
Serial.println("Dirección IP: ");
```

```
Serial.println(WiFi.localIP());
```

```
server.on("/", handleRoot);
```

```
server.on("/send", HTTP_POST,  
handleSend);
```

```

    server.on("/mode", HTTP_POST,
handleMode);

    server.on("/logout", HTTP_POST,
handleLogout);

server.begin();
Serial.println("Servidor web iniciado");

client.setServer(mqttServer, mqttPort);
client.setCallback(callback);

dht.begin();
pinMode(ldrPin, INPUT);
}

void loop() {
    if (!client.connected()) {
        reconnect();
    }
    client.loop();

    server.handleClient(); // Manejar las
solicitudes del servidor web

    unsigned long currentMillis = millis();

    // Enviar datos del DHT11 cada 1
minuto

    if (currentMillis - previousDHTMillis
>= DHTInterval) {
        previousDHTMillis = currentMillis;

```

```

        currentTemperature =
dht.readTemperature();

        if (!isnan(currentTemperature)) {
            Serial.print("Temperatura: ");
            Serial.println(currentTemperature);

            ubidots.add("temperature",
currentTemperature);

            ubidots.send("temperature"); // Envía
los datos de la variable "temperature" a
Ubidots
        }

        currentHumidity = dht.readHumidity();
        if (!isnan(currentHumidity)) {
            Serial.print("Humedad: ");
            Serial.println(currentHumidity);

            ubidots.add("humedad",
currentHumidity);

            ubidots.send("temperature"); // Envía
los datos de la variable "humedad" a
Ubidots
        }
    }

    // Controlar el servo en modo
automático

    if (!manualMode) {

        if (currentMillis - previousLDRMillis
>= ldrInterval) {

            previousLDRMillis = currentMillis;

            currentLDRValue =
analogRead(ldrPin);

```

```

    if (abs(currentLDRValue -
lastLDRValue) > thresholdHigh ||
abs(currentLDRValue -
lastSentLDRValue) > thresholdHigh/2) {

    int lumen = map(currentLDRValue,
0, 1023, 0, 1000); // Convertir valor del
LDR a lúmenes

    ubidots.add("lumen", lumen);

    ubidots.send("temperature"); //
Envía los datos de luz a Ubidots

    lastSentLDRValue =
currentLDRValue;

    }

    if (currentLDRValue > thresholdHigh)
{ // Si la luz es alta

    servo1.write(0); // Mover servo a 0
grados

    } else { // Si la luz es baja

    servo1.write(90); // Mover servo a
90 grados

    }

    }

}

```

// Enviar datos del servo cuando sea necesario

```

if (currentMillis - previousServoMillis
>= servoInterval) {

    previousServoMillis = currentMillis;

    currentServoValue = servo1.read();

    static int lastServoValue =
currentServoValue; // Declaración de la
variable lastServoValue

```

```

    if (currentServoValue !=
lastServoValue) {

        ubidots.add("servo_value",
currentServoValue); //

        ubidots.send("temperature"); // Envía
los datos del servo a Ubidots

        lastServoValue = currentServoValue;

    }

    Serial.print("Posicion actual del servo:
");

    Serial.println(currentServoValue);

    Serial.print("Fuente de la orden: ");

    Serial.println(manualMode ? "Manual"
: "Automático");

    Serial.print("Valor del LDR: ");

    Serial.println(currentLDRValue);

    Serial.println(currentTemperature);

    Serial.println(currentHumidity);

    }

}

```

Para este caso se le limito el numero de intentos para entrar a la página web, así como una contraseña más segura y método de cierre de sesión; en caso de no ser necesario la pagina web puedes cerrar el puerto.

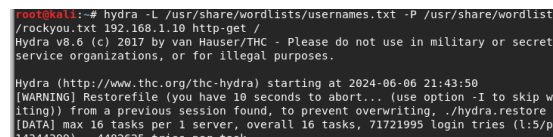


Figura 12.



Figura 13.

En la figura 12 se puede ver que nikto ya no detecta nada en el puerto http de la ip, y con nmap podemos ver que ya no hay puertos abiertos en el dispositivo.

Conclusiones

- 1) Inicialmente se logro implementar en la Kali-linux la red en la que esta conectada la placa esp-8266 haciendo uso del comando `dhclient -v` la cual es un protocolo que permite a un dispositivo conectarse a una red y obtener automáticamente una dirección ip única y la `-v` nos mostrara detalles sobre esto.
- 2) Luego usando herramientas como nmap se logro detectar un puerto disponible y vulnerable (puerto 80).
- 3) Después usando herramientas de análisis de vulnerabilidades (en este caso se utilizó nikto) en el puerto se encontraron varias vulnerabilidades, la mayoría relacionadas con la autenticación.
- 4) Luego de analizar las vulnerabilidades se implemento una forma de acceder a la placa (en este caso se accedió a la información alojada en el puerto 80) explotando estas vulnerabilidades encontradas, en este caso se logro obtener el usuario y contraseña para acceder al sitio.
- 5) Al final se implemento unas formas de protección al código de la placa para que esta no sea

fácilmente accesible a posibles ataques; las protecciones implementadas fueron la autenticación con contraseñas mas seguras, un numero limitado de intentos de acceder a este, una forma de cerrar sesión después de acceder al sitio y se implementaron unas cabeceras de protección, también se hizo sugerencias de utilizar una http segura la cual requiere de un certificado e implementar al código.

Referencias

Referencias

- [O. A. c. gpt, «chatgpt,» Open Ai, 2020. [En 1 línea]. Available: <https://chatgpt.com>.]
] [Último acceso: 05 2024].
- [cisco, «Práctica de laboratorio: escaneo 2 de puertos de un dispositivo IoT,» Cisco y
] / o sus afiliadas. Todos los derechos reservados. Cisco Public, 2018-2020.
- [k. s. service, «HYDRA - HERRAMIENTA DE 3 FUERZA BRUTA.,» 04 06 2021. [En línea].
] Available:
<https://www.kolibers.com/blog/hydra-herramienta-de-fuerza-bruta.html>.
[Último acceso: 05 2024].
- [D. Noguera, «Cabecera X-Frame-Options, 4 mejorar la seguridad Web,»
] WEBEMPRESA, 08 04 2020. [En línea].
Available:
<https://www.webempresa.com/blog/cabecera-x-frame-options-mejorar->

seguridad-web.html. [Último acceso: 05 2024].

[m. w. docs, «X-XSS-Protection,» 24 07
5 2023. [En línea]. Available:
] [https://developer.mozilla.org/es/docs/W
eb/HTTP/Headers/X-XSS-Protection](https://developer.mozilla.org/es/docs/Web/HTTP/Headers/X-XSS-Protection).
[Último acceso: 05 2024].

[m. w. docs, «X-Content-Type-Options,» 24
6 07 2023. [En línea]. Available:
] [https://developer.mozilla.org/es/docs/W
eb/HTTP/Headers/X-Content-Type-
Options](https://developer.mozilla.org/es/docs/Web/HTTP/Headers/X-Content-Type-Options). [Último acceso: 05 2024].